

SIMT: single instruction for multiple threads in one warp

#2

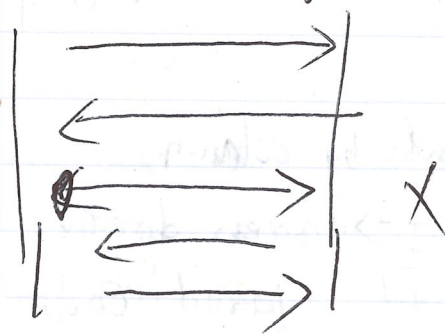
low precision improves arithmetic intensity

{ float32 $\Rightarrow$ intensity: 8 bytes/Flop $\leftarrow \text{ReLU}(x) = \max(0, x)$
float16 $\Rightarrow$ intensity: 4 bytes/Flop

lots of operations in modern GPUs are accelerated via low/mixed precision operations

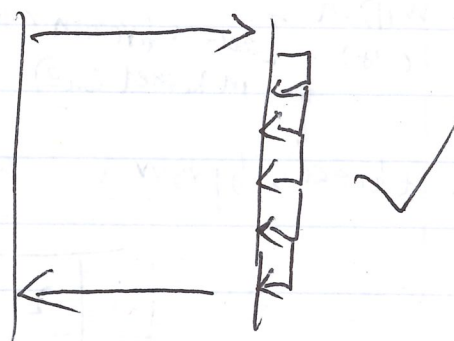
#3: operator fusion to minimize memory access

memory compute



naive (non-fused)

memory compute



(fused-kernel)

#4: re-computation: sacrifice compute for less memory access
 $\downarrow$ chips $\downarrow$ storage

store (retrieve): { params/gradients
activation/hidden
optimizer states
loss values

$\rightarrow$ remove activations, store for re-computing them

→ access on byte space, return a chunk of bytes

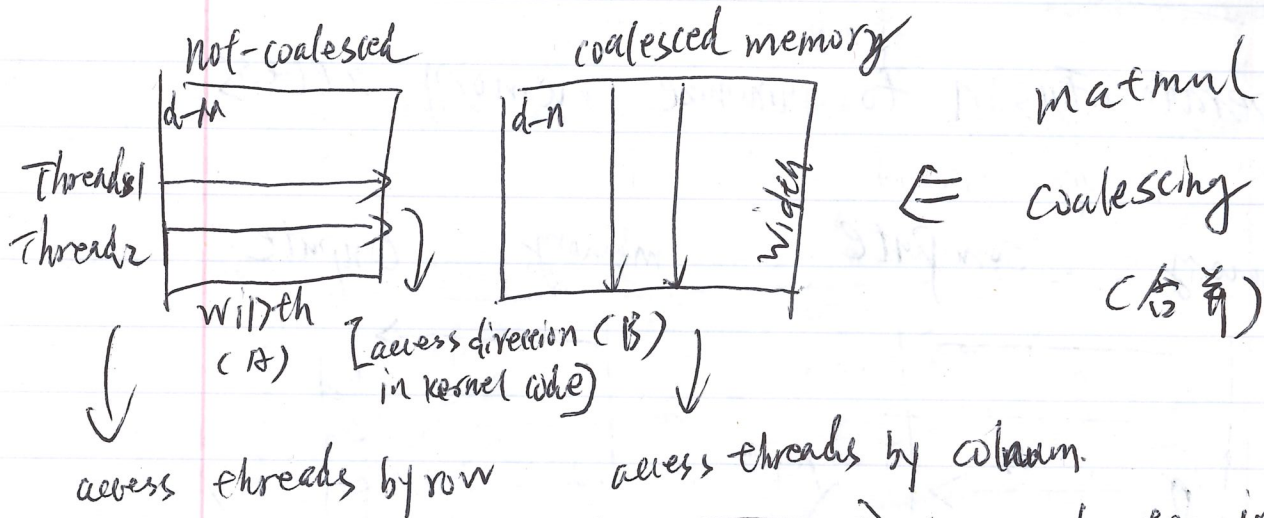
#5: memory ~~code~~ coalescing and iDRAM

iDRAM: global memory (dynamic memory)

↳ is read in 'burst mode' - each read gives you many bytes



For row-major matrices - threads that move along rows are not coalesced.



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Access direction in kernel code

